

VITALS-Edge

Vehicle Intelligent Telemetry & Anomaly-Learning System

An edge-first predictive-maintenance system that detects incipient engine and rotating-machinery faults **on the device**, estimates Remaining Useful Life in real time, and ships only kilobyte-scale alerts to a thin cloud dashboard.

Project concept & technical dossier

Team Algorithm404 · Department of AI & Data Science

Sri Eshwar College of Engineering, Coimbatore

Working name — swap freely. This dossier sharpens the earlier draft into a buildable, demoable, edge-first submission.

1 Executive Summary

Unplanned mechanical failure is expensive and dangerous: in a test cell it halts validation programs, and on the road it strands vehicles and creates safety risk. Traditional maintenance is either run-to-failure or fixed-interval — the first wastes assets, the second wastes money. VITALS-Edge closes that gap with a single, sharply-scoped idea: **run the fault-detection and Remaining Useful Life (RUL) models directly on an on-asset edge computer**, so a developing bearing, injector, or cooling fault is caught in under a second, offline, before it becomes catastrophic.

The system fuses vibration, temperature/pressure, and CAN/OBD-II signals on a compact edge box (an NVIDIA Jetson Orin Nano Super), scores each short window of data with a compact anomaly detector plus a CNN-LSTM RUL model, and takes a decision locally. Only a small alert packet — fault type, severity, health score, projected RUL — travels to a thin cloud layer that hosts a lightweight digital twin and a dashboard. This keeps latency low, keeps raw data private, and keeps the design honest to the theme: the intelligence genuinely lives *at the edge*, not in the cloud.

Crucially for a hackathon, the entire pipeline is demonstrable **without a live engine**. We train on public run-to-failure datasets (NASA C-MAPSS, CWRU bearing, AI4I 2020) and replay them as a real-time sensor stream into the exact edge service that would run in the field. The judges see a real model, on real edge hardware, producing real-time alerts and an RUL curve that counts down to failure.

2 What Changed From the First Draft (and Why)

The earlier write-up was broad and ambitious, which is a liability under judging criteria that reward **technical feasibility** and a **working live prototype**. The table below is the honest diff — each change trades scope for a demo you can actually stand behind.

Area	First draft	This version — and the reasoning
Scope	Test-cell + in-service + CV + LLM assistant + AR, all at once	One hero use case: engine / rotating-machinery health via vibration + CAN. CV, LLM assistant and in-service fleet are explicit <i>roadmap</i> , not MVP. Narrow scope = credible demo.
Where AI runs	Heavy cloud + Unity/cloud digital twin; edge mostly forwards data	Inference runs on the edge box. Cloud is thin (broker + TSDB + dashboard + a small twin). This is what 'AI at the Edge' literally asks for.
Demo path	Assumes instrumented engine + vehicle install	Dataset-replay harness feeds the real edge service, so the system is fully demoable on a laptop + Jetson with zero engine access.
Edge hardware	Jetson Orin Nano listed at 21 TOPS	Corrected to Jetson Orin Nano Super — 67 INT8 TOPS, 8 GB, US\$249 , upgradable via JetPack; India stock and a 'Made-in-India' deployment kit exist.
Evidence	Placeholder / internally-numbered citations, some future-dated	Replaced with real, checkable references (NVIDIA, NASA PCoE, MDPI, arXiv). Named partners (Emerson, AWS) used as strategic hooks.

3 Reading the Brief: What InnoVent-27 Actually Rewards

InnoVent-27's theme is **AI at the Edge** — localized, real-time intelligence for automotive, heavy machinery and aerospace — and predictive maintenance and digital twins are named target applications. Judging runs in three stages (concept screening → virtual proof-of-concept → live prototype demo) and weighs innovation, technical feasibility, practical applicability, diversity of thought, and industry impact. Two design consequences follow directly:

- **Edge-first is not optional.** A submission where the model runs in the cloud reads as off-theme. Our differentiator must be that detection happens on-device, sub-second, and survives loss of connectivity.
- **The prototype must be real by the final stage.** That is why the dataset-replay harness matters: it de-risks the demo so a working system exists even if physical instrumentation slips.

Strategic hook — use the partners

Emerson's Test & Measurement business (built on NI technology) is a named partner, and the test-cell / engine-dyno framing sits squarely in their world — the same domain Team Algorithm404 already worked in on LeakSense Twin. AWS is the other partner, so the thin cloud layer can be positioned on AWS IoT Core + Timestream for the 'productionize' slide. Aligning the story with the sponsors is cheap and scores on practical applicability.

4 Sharpened Problem Statement

Rotating machinery and combustion engines accumulate damage — bearing wear, imbalance, injector drift, cooling degradation — long before a hard failure is visible. On-board diagnostics today are tuned for emissions and stored fault codes, not early prognosis, and high-rate signals that *do* carry the early warning (vibration spectra, load-normalized temperatures) are rarely analyzed in real time. The result is late detection, unplanned downtime, and avoidable secondary damage.

VITALS-Edge target: detect an incipient fault and issue a health-status change in under two seconds, on the asset, and translate the trend into a Remaining Useful Life estimate accurate enough to schedule service before failure — first for a test-cell engine / rig (primary), then for in-service vehicles (roadmap), on a unit that costs a few hundred dollars and needs no permanent cloud link.

5 Objectives & Measurable KPIs

KPI	Definition	MVP target (demo)
Fault detection accuracy	Precision / recall on seeded & dataset faults	$\geq 90\%$ recall, $\geq 90\%$ precision
Edge-to-alert latency	Window arrival → local alert raised	< 2 s (target < 300 ms inference)
RUL error	RMSE vs. ground-truth on C-MAPSS FD001	Competitive with LSTM baseline (~15–20 cycles)
False-alarm rate	False alerts per hour of nominal operation	< 1 per demo session; $< 1 / 1000$ h goal
Data reduction	Bytes to cloud vs. raw sensor volume	Alert packet < 1 KB vs. MB/s raw
Offline resilience	Behaviour with no network	Full local detection + buffered alerts

6 System Architecture (Edge-First)

Three thin lanes: sensing on the asset, an edge AI box that does all the thinking, and a deliberately small cloud layer. The heavy arrows carry raw signals only *inside* the edge box; only compact alert packets cross to the cloud, and the sole return path is occasional over-the-air model/config updates.

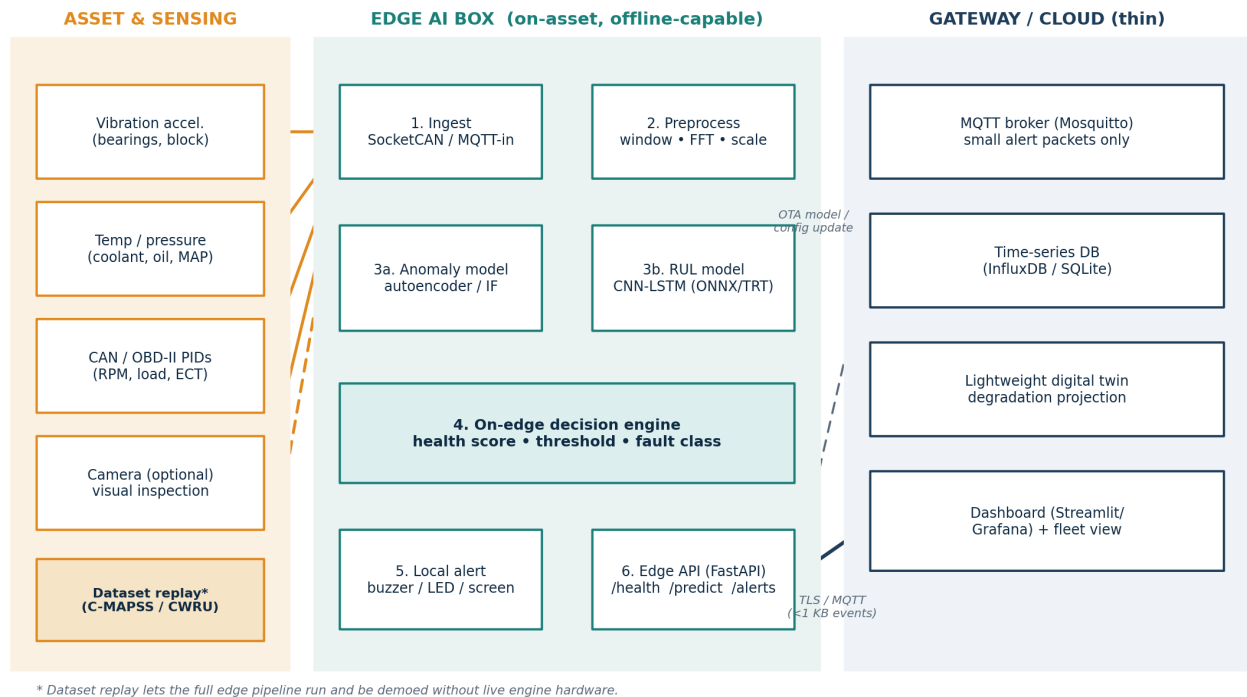


Figure 1 — Edge-first architecture. All inference runs on the edge box; the cloud is a thin broker + time-series store + lightweight digital twin + dashboard.

6.1 Edge box — the core

- **Ingest.** SocketCAN reads OBD-II PIDs (RPM, load, coolant temp, MAP); an MQTT-in / serial path reads the vibration and temp/pressure sensors. In demo mode this input is fed by the replay harness.
- **Preprocess.** Sliding windows, FFT / envelope spectrum for vibration, load-normalization and scaling — the same transforms used at training time, shared as one code path to avoid train/serve skew.
- **Two models.** (a) an unsupervised anomaly detector (autoencoder or isolation forest) that flags 'this doesn't look normal' without needing labelled faults; (b) a CNN-LSTM RUL regressor that projects cycles-to-failure. Both exported to ONNX and, on Jetson, TensorRT INT8 for speed.
- **Decision engine.** Combines anomaly score, RUL trend and thresholds into a single health state (Healthy / Watch / Warning / Critical) and a fault class where known.
- **Outputs.** A local alert (buzzer / LED / on-box screen) for connectivity-independent safety, plus a small FastAPI service exposing /health, /predict and /alerts — reusing the team's existing FastAPI backend experience.

6.2 Thin cloud layer

A Mosquitto MQTT broker receives sub-kilobyte events; InfluxDB (or SQLite for the demo) stores the health time-series; a lightweight digital twin projects the degradation curve forward for 'what-if' service scheduling; and a Streamlit or Grafana dashboard shows live health, the RUL countdown, and a fleet view. For the production slide this maps cleanly onto AWS IoT Core + Timestream + Managed Grafana.

7 Executable Build & Demo Pipeline

This is the concrete, runnable flow — the part that turns a concept deck into a working prototype. Steps A–C happen offline on a laptop or Colab; steps D–F are the live edge demo. Nothing here depends on owning an instrumented engine.

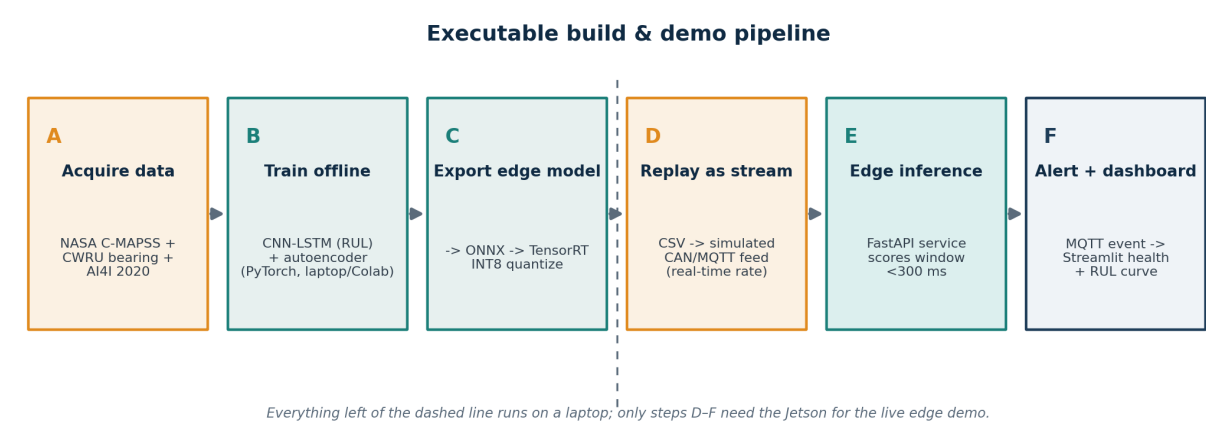


Figure 2 — The build/demo pipeline. Train offline on public data, export a compact edge model, then replay recorded runs as a real-time stream into the on-device service.

7.1 Component stack

Layer	Tools	Notes
Training	Python, PyTorch / TensorFlow, scikit-learn, NumPy/SciPy	Runs on laptop/Colab; C-MAPSS trains in minutes on CPU.
Edge runtime	ONNX Runtime / TensorRT, FastAPI + Uvicorn, paho-mqtt	Same service on laptop (CPU) or Jetson (TensorRT INT8).
Signal DSP	NumPy FFT, librosa/scipy envelope, sliding windows	Bearing fault frequencies from vibration spectra.
Messaging	MQTT (Mosquitto), TLS; REST for on-demand	MQTT suits frequent small IoT events.
Storage	InfluxDB / SQLite (demo), Parquet logs	Time-series health + raw buffer for replay.
Dashboard	Streamlit or Grafana; React optional	Live health, RUL curve, fleet map, twin view.
Replay harness	Python generator that streams CSV rows at real-time rate	The key that makes the demo hardware-optional.

8 Machine-Learning Approach

8.1 Anomaly detection (unsupervised)

Because labelled fault data is scarce, the first line of defence is unsupervised: an autoencoder (or isolation forest) trained only on healthy operation. When a bearing starts to fail, reconstruction error rises and the health score drops — no fault labels required. This catches *unknown* failure modes, which is exactly what a real deployment faces.

8.2 Remaining Useful Life (supervised)

For RUL we use a CNN-LSTM: 1-D convolutions extract local degradation features from each sensor window and the LSTM captures how those features trend over time, with a linear output for cycles-to-failure. LSTM-family models are the standard, well-understood baseline on C-MAPSS and train fast enough on a CPU, so they are a low-risk MVP choice; a Transformer or CNN-BiLSTM-attention variant is a credible 'stretch' upgrade if time allows. Loss is MSE (or an asymmetric loss that penalizes late predictions more, since a missed failure is worse than an early warning).

8.3 Edge optimization & evaluation

- Export to ONNX, then TensorRT INT8 quantization on Jetson for low-millisecond inference at low power.
- Classification metrics: accuracy, precision, recall, F1, ROC-AUC. RUL metrics: RMSE and the standard asymmetric scoring function used on C-MAPSS. Operational metrics: time-to-detection, false-alarm rate.
- Monthly / triggered retraining: engineer-reviewed anomalies feed back as new labels — a self-improving loop.

9 Datasets & Data Plan

Public run-to-failure datasets give us real degradation signals to train and demo on immediately; seeded faults on a small bench rig (imbalance, loosened mount) provide live validation later.

Dataset	What it is	Used for
NASA C-MAPSS	Turbofan run-to-failure sim; FD001–FD004, 21 sensors + 3 op settings	RUL model + the countdown demo (de-facto benchmark)
CWRU bearing	Drive-/fan-end bearing faults, 12 & 48 kHz vibration	Vibration anomaly + fault-class demo
IEEE PHM 2012	Accelerated bearing life test (FEMTO)	Alternative bearing RUL benchmark
AI4I 2020	10k synthetic IIoT records, 14 features, 5 failure types	Fast multi-label fault-classification baseline
Bench rig (own)	Motor + flywheel with seeded imbalance / loose bolts	Live validation of the edge pipeline

10 Hardware & Bill of Materials (corrected)

Prototype BOM for one edge unit. The demo itself runs on a laptop; the Jetson is what makes the 'genuinely at the edge' claim tangible in front of the jury.

Component	Example part	~Price	Note
Edge compute	NVIDIA Jetson Orin Nano Super Dev Kit	\$249	67 INT8 TOPS, 8 GB, 102 GB/s; India stock available
Low-cost alt.	Raspberry Pi 5 (8/16 GB) + Coral TPU	~\$80–120	No CUDA; fine for anomaly model, weaker for CV

Vibration	ADXL1001 / ADXL335 MEMS accelerometer	\$10–70	High-freq. bearing monitoring
Temp / press.	DS18B20, BMP390	~\$8	Coolant/oil temp, MAP
CAN / OBD-II	ELM327 or MCP2515 CAN board	\$8–15	Read engine PIDs via SocketCAN
Camera (opt.)	Raspberry Pi Camera Module 3	~\$30	Visual inspection — roadmap
Power / misc	12V→5V buck, enclosure, wiring, SD	~\$40	Vehicle-grade power
Total	Per edge unit (excl. labour)	~\$350–450	Well within hackathon budget

11 The Live Demo (mapped to the three judging stages)

Stage	Deliverable	What the judges see
1. Concept screening	This dossier + concept deck	Sharp edge-first problem/solution, feasibility, partner alignment
2. Virtual PoC	Recorded pipeline run + notebook	Trained models, metrics on C-MAPSS/CWRU, replay feeding the edge API
3. Live prototype	Jetson + dashboard, live	Real-time health scoring; a seeded/replayed fault trips a <2 s alert; RUL curve counts down; network unplugged and it still detects locally

The money shot

Mid-demo, pull the network cable. The dashboard freezes — but the edge box keeps scoring and the on-box LED still flips to **Warning** when the replayed bearing fault develops. That single moment proves 'AI at the Edge' better than any slide.

12 Timeline (feasibility-first)

Phase	Weeks	Focus	Milestone
1 · Foundation	1–2	Data pipeline, C-MAPSS/CWRU loaded, replay harness, FastAPI skeleton	Replay stream hits API
2 · Models	3–5	Train anomaly + CNN-LSTM RUL; validate metrics; ONNX export	Metrics meet KPI targets
3 · Edge	6–8	Deploy to Jetson (TensorRT), decision engine, local alert, MQTT	End-to-end <2 s on device
4 · Cloud + twin	9–10	Broker, TSDB, dashboard, degradation twin	Live dashboard + RUL curve
5 · Harden + rig	11–12	Bench rig seeded faults, offline test, demo rehearsal	Full live demo rehearsed

13 Risks & Mitigation

Risk	Mitigation
No engine / instrumentation access	Dataset-replay harness makes the whole system demoable on a laptop + Jetson.
Fault-data scarcity	Unsupervised anomaly detection + synthetic/seeded faults + public run-to-failure data.
Model too heavy for edge	Compact CNN-LSTM, INT8 quantization, TensorRT; fall back to Pi + anomaly-only if needed.
Scope creep (CV, LLM, AR)	Explicitly parked as roadmap; MVP is vibration + CAN health only.
Demo fragility on stage	Pre-recorded PoC as backup; deterministic replay; rehearse the network-unplug moment.
Connectivity gaps in field	Edge buffers alerts and raises local alarms; syncs when a link returns.

14 Why This Wins

- **Truly edge.** Inference and the safety decision happen on-device and survive a network cut — the theme, demonstrated literally, not asserted.
- **Feasible and demoable.** The replay harness removes the single biggest hackathon risk (no hardware) while still showing real hardware doing real inference.
- **Two-layer safety net.** Unsupervised anomaly detection catches unknown faults; supervised RUL turns trends into an actionable service date.
- **Partner-aligned.** Test-cell framing fits Emerson; the thin cloud maps to AWS — practical applicability the jury can see.
- **Team fit.** Builds on Algorithm404's LeakSense Twin test-cell work and existing FastAPI backend skill, so the plan is credible for *this* team.

15 References

- 1 Tata Technologies. *InnoVent-27 — AI at the Edge* (theme, categories incl. 3.2.1.3; partners Emerson & AWS; three-stage judging). tatatechnologies.com/in/innovent, 2026.
- 2 NVIDIA. *Jetson Orin Nano Super Developer Kit* — 67 INT8 TOPS, 8 GB, 102 GB/s, US\$249; JetPack upgrade path. developer.nvidia.com/embedded/buy-jetson, 2024–2026.
- 3 Saxena, A. & Goebel, K. *Turbofan Engine Degradation Simulation Data Set (C-MAPSS)*. NASA Ames Prognostics Data Repository, 2008.
- 4 Case Western Reserve University Bearing Data Center. *Bearing vibration dataset* (12 & 48 kHz, drive-/fan-end faults).
- 5 Nectoux, P. et al. *PRONOSTIA / FEMTO IEEE PHM 2012 bearing challenge dataset*.
- 6 Matzka, S. *AI4I 2020 Predictive Maintenance Dataset*. UCI Machine Learning Repository, 2020.
- 7 Yildirim, U. & Afser, H. "Linear Methods for Predictive Maintenance: the Case of NASA C-MAPSS Datasets." *Applied Sciences* 15(18):9945, MDPI, 2025.
- 8 Recent CNN-LSTM / BiLSTM-attention RUL studies on C-MAPSS FD001 (arXiv, 2024–2026) — model-family baselines.
- 9 HiveMQ. *MQTT vs HTTP for IoT* — messaging efficiency for frequent small events (technical blog).

Live-web figures (hardware price/specs, dataset descriptions, hackathon details) were verified at the time of writing; confirm current numbers before final submission.